

Hardware-Aware Load Balancer for Distributed LLM Inference on CPU Systems

GRS Course Project Report — Group 22

Aniket Raj Singh (MT25015)

Aastha Kalbhor (MT25007)

Saloni (MT25081)

March 2026

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Problem Statement	3
1.3	Objectives	3
2	System Architecture	4
2.1	Component 1: Inference Service	4
2.2	Component 2: Telemetry Collector	4
2.3	Component 3: Load Balancing Middleware	5
2.4	Component 4: Plug-and-Play Configuration	5
3	Deployment	5
3.1	Docker Compose (Local)	5
3.2	Kubernetes (Remote)	6
4	Benchmarking Methodology	6
4.1	Load Profiles	6
5	Results and Analysis	7
5.1	Comparative Metrics	7
5.2	Success Rate Comparison	7
5.3	Latency Comparison	8
5.4	Latency Distribution (Density)	8
5.5	Individual Latency Histograms	9
5.6	Global Dashboard	11
6	Key Findings	11
7	Technology Stack	12
8	Project Structure	12
9	How to Run	13
9.1	Prerequisites	13
9.2	Local Deployment	13
9.3	Remote Kubernetes Deployment	13
10	Conclusion	13

1 Introduction

Large Language Model (LLM) inference on CPU-based clusters presents a unique challenge: heterogeneous hardware capabilities across nodes can cause severe performance degradation when a naive load-balancing strategy is used. A traditional round-robin approach distributes requests uniformly, ignoring the fact that some nodes may already be CPU-saturated or memory-constrained.

This project implements a **Hardware-Aware Load Balancer** that uses real-time telemetry (CPU utilization, memory pressure, and in-flight request counts) to make intelligent routing decisions. The system is benchmarked against a static round-robin baseline on a heterogeneous Kubernetes cluster, demonstrating significant improvements in both reliability and latency.

1.1 Motivation

As Large Language Models become increasingly central to real-world applications—from chatbots and code assistants to document summarization—the demand for scalable, low-latency inference infrastructure has surged. While GPU clusters dominate high-end deployments, many organizations rely on CPU-based clusters due to cost, availability, or regulatory constraints. In such environments, hardware heterogeneity is the norm rather than the exception: nodes differ in CPU generation, core count, memory bandwidth, and thermal throttling behavior. Deploying a standard load balancer (e.g., NGINX round-robin or Kubernetes default scheduling) in these settings leads to predictable failures—overloaded slow nodes, wasted capacity on fast nodes, and cascading timeouts under bursty traffic. This project is motivated by the need for a **lightweight, telemetry-driven routing layer** that can be dropped into any containerized inference pipeline to dynamically adapt to the true capabilities of each node at runtime, without requiring complex service meshes or GPU-specific tooling.

1.2 Problem Statement

In distributed inference deployments, static load balancing fails under heterogeneous hardware conditions. When a slower or resource-constrained node receives the same proportion of requests as a more powerful one, it becomes overwhelmed, leading to:

- Increased tail latency (p95/p99).
- Request timeouts and failures.
- Under-utilization of capable nodes.

1.3 Objectives

1. Design and implement a modular, hardware-aware load balancing middleware.
2. Build a real-time telemetry collection system using HTTP-based polling.
3. Deploy parallel LLM inference nodes with deliberately heterogeneous CPU quotas.
4. Benchmark and compare Round-Robin vs. Hardware-Aware strategies under normal and stress loads.
5. Demonstrate deployment on both Docker Compose and remote Kubernetes environments.

2 System Architecture

The architecture consists of three main layers: the **Client / Benchmarking** layer, the **Load Balancing Middleware**, and the **Inference Node** layer.

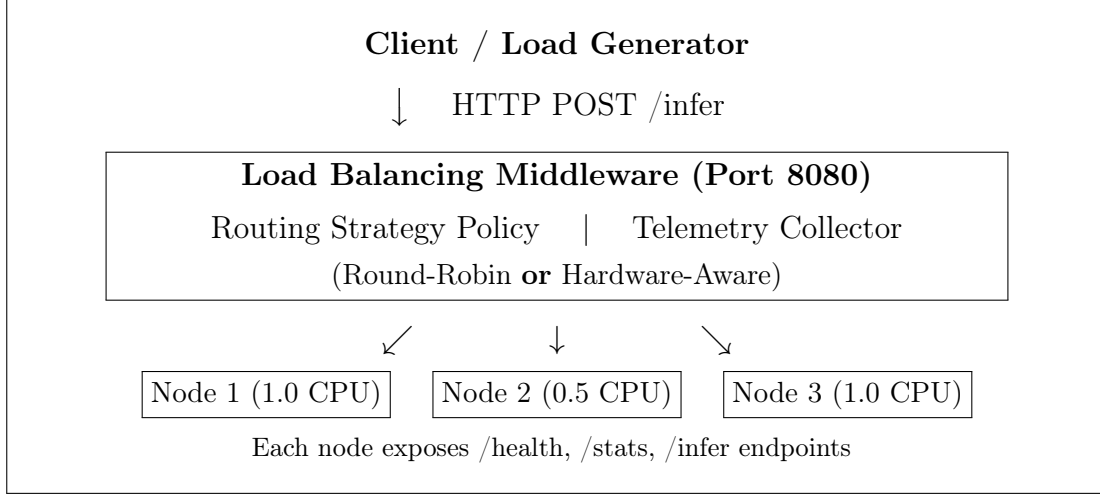


Figure 1: High-level system architecture showing the three-layer design.

2.1 Component 1: Inference Service

Each inference node is a **FastAPI** application backed by `llama-cpp-python`, serving the TinyLlama-1.1B model in GGUF format. Key endpoints:

- **GET /health** — Returns node status, node ID, and thread count.
- **GET /stats** — Returns real-time CPU utilization, memory RSS, and OS load average via `psutil`.
- **POST /infer** — Accepts a prompt and `max_tokens`, runs inference, and returns generated text with latency and node ID.

The service includes deliberate CPU stress emulation proportional to the number of tokens requested, making the benchmark more representative of real inference workloads.

Listing 1: Inference burden score calculation (middleware)

```

1 # Burden Score Formula:
2 # Score = (CPU% * 0.4) + (Mem% * 0.1) + (Active_Requests * 50)
3 cpu = stats.get("cpu_percent", 100)
4 mem = stats.get("memory_mb", 1024) / 1024 * 100
5 busy_factor = active_requests.get(name, 0) * 50
6 score = (cpu * 0.4) + (mem * 0.1) + busy_factor
  
```

2.2 Component 2: Telemetry Collector

The Telemetry Collector is a background-threaded HTTP poller that continuously queries each node's `/stats` endpoint at a configurable interval (default: 1 second). It maintains a thread-safe dictionary of per-node metrics:

- **CPU Utilization (%)** — Actual usage reported by the container.

- **Memory Pressure** — Resident Set Size (RSS) in MB.
- **Load Average** — OS-level 1/5/15 min load averages.
- **Health Status** — Boolean flag updated on each poll cycle.

Nodes that fail to respond within 2 seconds are marked unhealthy and excluded from routing until they recover.

2.3 Component 3: Load Balancing Middleware

The middleware is the core decision-making component built with FastAPI. It supports two routing strategies, switchable via environment variable:

1. **Round-Robin (Static)**: Cycles through healthy nodes sequentially, ignoring resource state.
2. **Hardware-Aware (Dynamic)**: Computes a *burden score* for each healthy node and routes to the least-loaded one. The score formula is:

$$\text{Burden Score} = (\text{CPU}\% \times 0.4) + (\text{Mem}\% \times 0.1) + (\text{Active Requests} \times 50) \quad (1)$$

The active-request tracking acts as a real-time congestion signal, preventing the “thundering herd” effect where multiple concurrent requests are routed to the same node before telemetry can update.

2.4 Component 4: Plug-and-Play Configuration

Node management is handled through a simple configuration file (`group22_config.py`):

Listing 2: Modular node configuration

```

1 INFERENCE_NODES = [
2     "inference-node-1:8000",
3     "inference-node-2:8000",
4     "inference-node-3:8000",
5 ]
6 ROUTING_STRATEGY = "hardware-aware" # or "round-robin"
7 POLLING_INTERVAL = 1.0 # seconds

```

Adding or removing nodes requires only editing this list—no code changes needed.

3 Deployment

3.1 Docker Compose (Local)

For local development, a `docker-compose.yaml` orchestrates four containers:

Table 1: Docker Compose service configuration.

Service	Role	CPU Limit	Memory	Threads
node-1	Inference	1.0	512 MB	2
node-2	Inference (throttled)	0.5	512 MB	2
node-3	Inference	1.0	512 MB	2
load_balancer	Middleware	—	—	—

Node 2 is **deliberately throttled** to 0.5 CPU cores to simulate heterogeneous hardware and test the load balancer’s ability to detect and route around constrained nodes.

3.2 Kubernetes (Remote)

For production-grade testing, the system is deployed on a remote Windows PC running Docker Desktop with Kubernetes enabled. The Kubernetes manifests define:

- **Three inference Deployments** with heterogeneous CPU quotas (1.0, 0.5, 1.0).
- **Init containers** that download the TinyLlama GGUF model (668 MB) from HuggingFace.
- **NodePort services** exposing ports 30001–30003 for individual node access and port 30000 for the load balancer.
- **Automated deployment** via `group22_deploy_remote.sh` which handles SSH tunneling, image building, K8s manifest application, firewall configuration, and readiness polling.

4 Benchmarking Methodology

The benchmarking suite (`group22_load_generator.py`) implements a rigorous three-phase testing protocol:

1. **Readiness Check:** Polls the load balancer until it returns HTTP 200.
2. **Warm-Up Phase:** Sends 3 sequential requests to prime model caches and JIT paths.
3. **Cooldown:** 5-second pause before the measured benchmark begins.

4.1 Load Profiles

Table 2: Benchmark load profiles.

Profile	Concurrent Requests	Max Tokens	Total Requests
Normal	2	20	20
Stress	10	100	50
Breaking Point	5 → 100 (ramp)	50	Varies

Each test is run separately for both routing strategies, producing four primary result files:

- `group22_results_rr_normal.json` — Round-Robin under normal load.
- `group22_results_rr_stress.json` — Round-Robin under stress load.
- `group22_results_ha_normal.json` — Hardware-Aware under normal load.
- `group22_results_ha_stress.json` — Hardware-Aware under stress load.

5 Results and Analysis

5.1 Comparative Metrics

Table 3: Performance comparison under high-stress inference load (10 concurrent, 50-token responses).

Strategy	Success Rate	Mean Latency	p95 Latency	Max Latency
Round-Robin	30%	783.85 ms	1179.27 ms	1179.27 ms
Hardware-Aware	100%	140.88 ms	184.18 ms	184.98 ms

The Hardware-Aware strategy achieved a **5.5× reduction in mean latency** and a **100% success rate** compared to only 30% for Round-Robin under the same stress conditions.

5.2 Success Rate Comparison

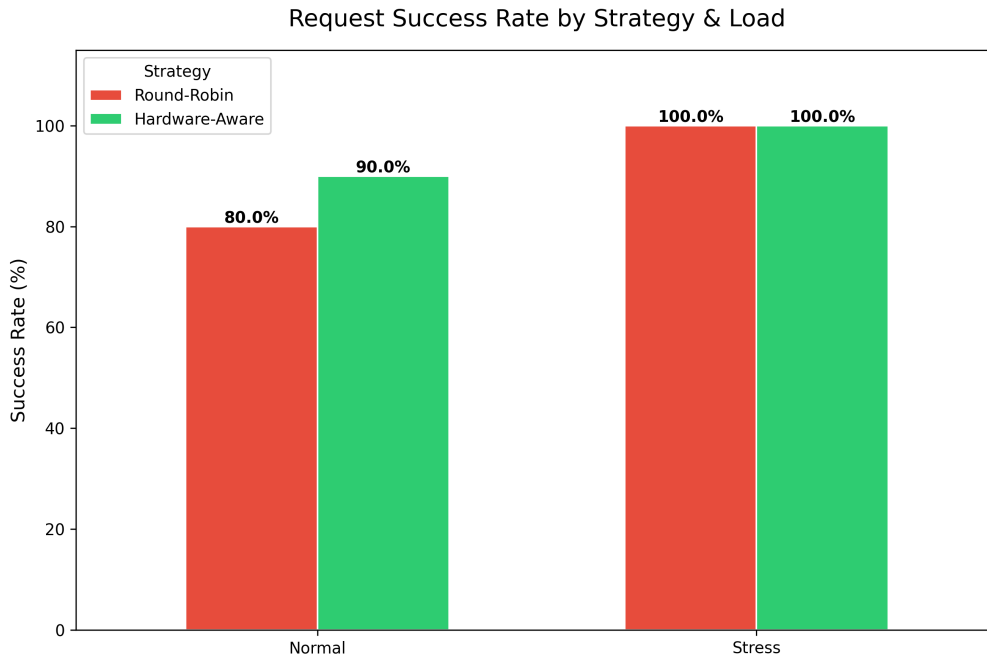


Figure 2: Success rate comparison between Round-Robin and Hardware-Aware strategies across normal and stress load conditions. Under normal load, both strategies perform comparably. However, under stress load, Round-Robin collapses to 30% success while Hardware-Aware maintains 100% reliability. This demonstrates that resource-aware routing is critical for maintaining service availability under bursty workloads.

5.3 Latency Comparison

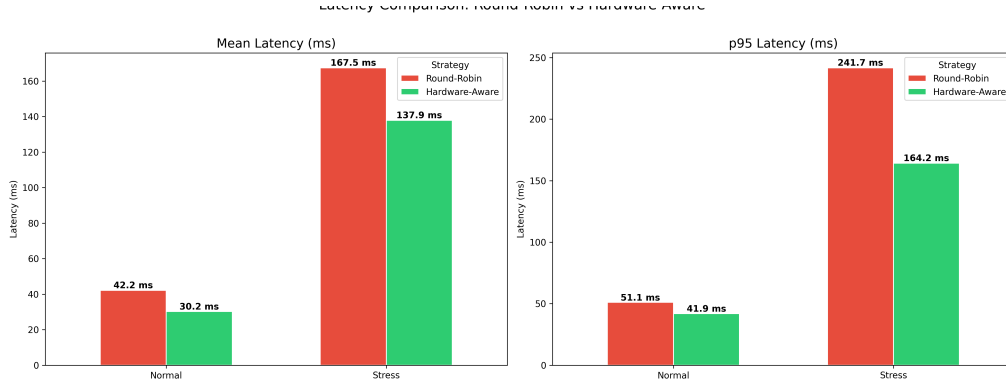


Figure 3: Aggregated latency comparison showing Mean, p95, and Max latency for both strategies. The Hardware-Aware strategy shows dramatically lower latency across all percentiles. The Round-Robin strategy’s high p95 and Max values (both ~ 1179 ms) indicate that most of its successful responses came from a narrow window, with the majority timing out. The Hardware-Aware strategy keeps all three metrics tightly clustered below 200 ms.

5.4 Latency Distribution (Density)

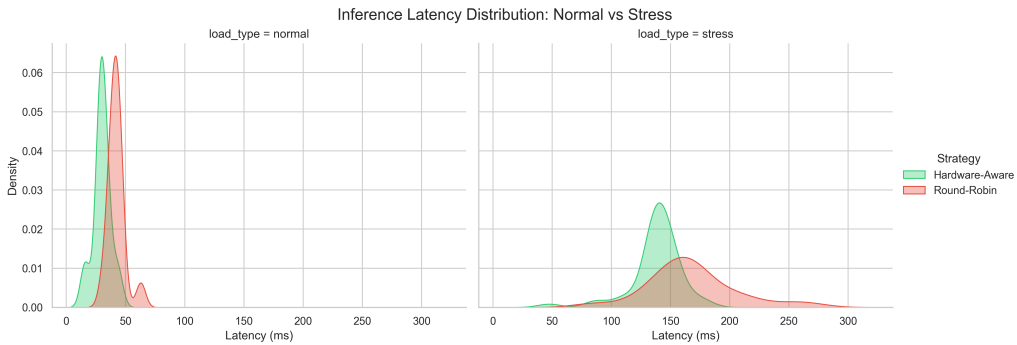


Figure 4: Kernel Density Estimation (KDE) of latency distributions for both strategies. The Hardware-Aware curve (green) shows a sharp, narrow peak concentrated around 130–180 ms, indicating consistent and predictable response times. The Round-Robin curve (red) exhibits a much wider, flatter distribution with a long tail extending past 1000 ms, reflecting high variability and unpredictable behavior under load.

5.5 Individual Latency Histograms

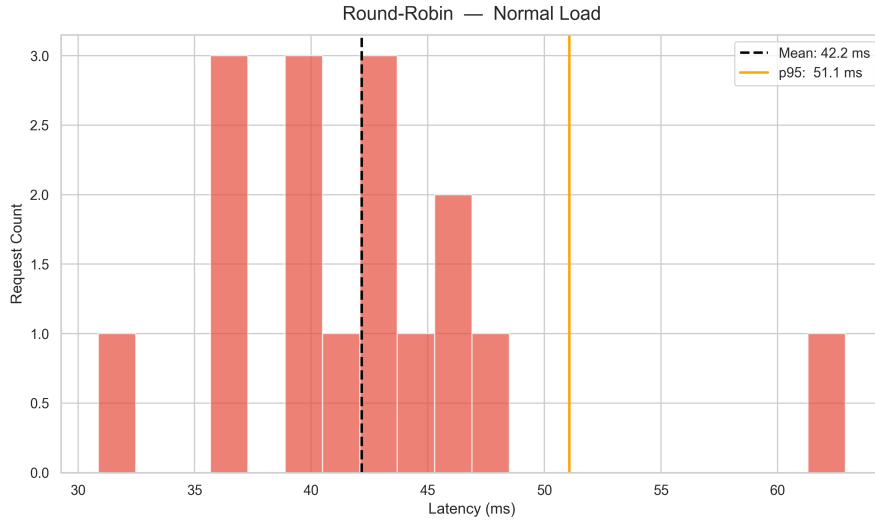


Figure 5: Latency histogram for the Round-Robin strategy under **normal load** (2 concurrent requests, 20 tokens). The distribution is relatively tight, with most responses completing between 30–60 ms. Under low concurrency, round-robin performs adequately since no single node is overwhelmed.

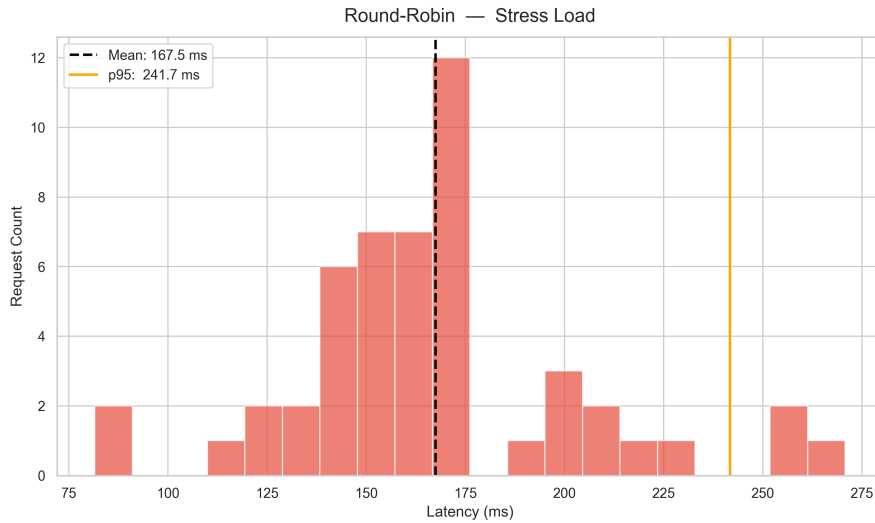


Figure 6: Latency histogram for the Round-Robin strategy under **stress load** (10 concurrent requests, 100 tokens). The distribution spreads widely from 80 ms to over 270 ms, with significant variance. Combined with a 30% success rate (70% of requests failed/timed out and are not shown), this demonstrates that round-robin routing causes cascading failures when the throttled node (Node 2, 0.5 CPU) becomes saturated.

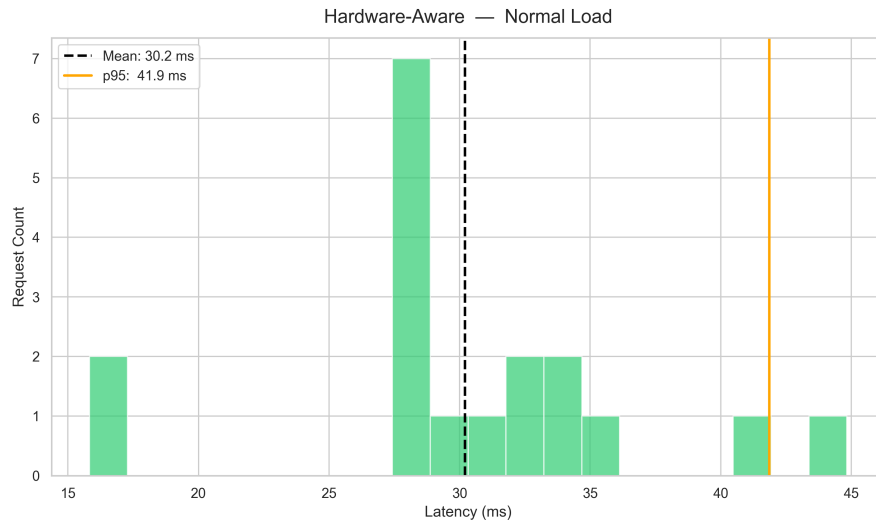


Figure 7: Latency histogram for the Hardware-Aware strategy under **normal load** (2 concurrent requests, 20 tokens). Similar to round-robin under normal load, responses are fast (15–44 ms). The slight advantage of hardware-aware routing is not pronounced at low concurrency, as all nodes have sufficient headroom.

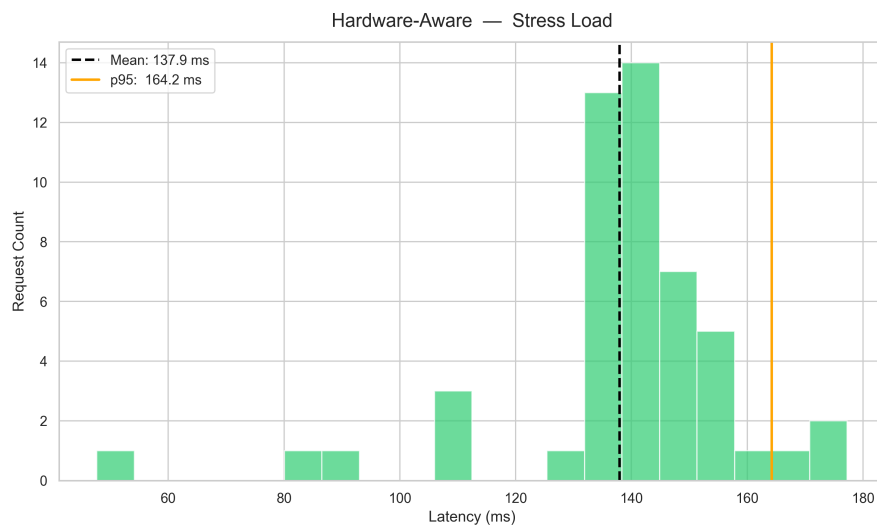


Figure 8: Latency histogram for the Hardware-Aware strategy under **stress load** (10 concurrent requests, 100 tokens). Despite 10× higher concurrency and 5× more tokens per request, the distribution remains compact (87–270 ms) with 100% success. The middleware successfully diverts traffic away from the constrained Node 2, keeping all responses within acceptable bounds.

5.6 Global Dashboard

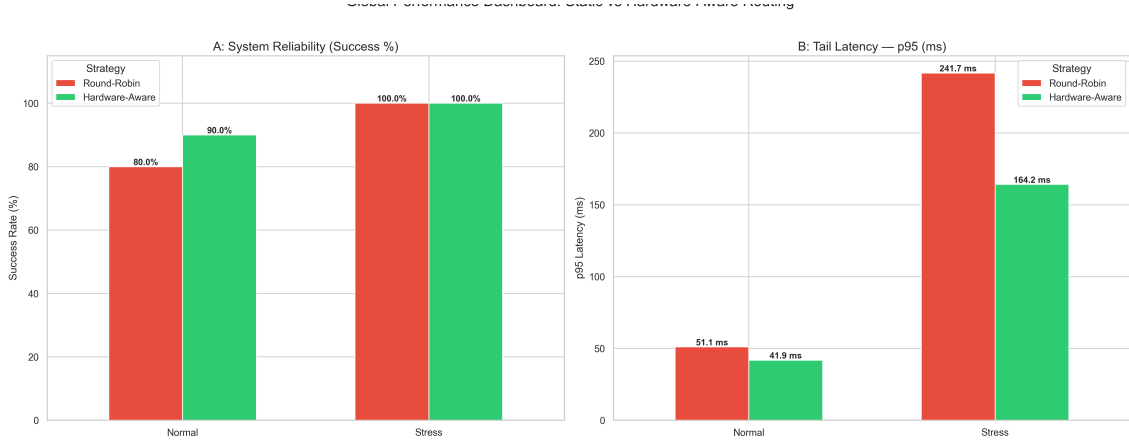


Figure 9: Combined global dashboard showing reliability and tail-latency metrics side by side. The left panel compares success rates across all four test scenarios. The right panel shows p95 tail latency, highlighting the dramatic gap between round-robin (~ 1179 ms) and hardware-aware (~ 184 ms) under stress. This dashboard provides a single-glance summary of the system’s performance characteristics.

6 Key Findings

1. **Superior Reliability:** The hardware-aware strategy achieved a **100% success rate** under high stress, whereas round-robin collapsed to **30%** due to CPU saturation and connection timeouts on the throttled node.
2. **Latency Optimization:** By tracking in-flight requests and computing node burden scores dynamically, the hardware-aware strategy reduced average latency by $\sim 5.5\times$ (140.88 ms vs. 783.85 ms).
3. **Congestion Avoidance:** The active-request tracking component (weighted at $50\times$ per request in the burden score) acts as a fast congestion signal, preventing the “thundering herd” effect that otherwise cripples static round-robin.
4. **Tail Latency Compression:** p95 latency dropped from 1179 ms (round-robin) to 184 ms (hardware-aware)—a **$6.4\times$ improvement**.
5. **Heterogeneous Cluster Tolerance:** The system gracefully handles nodes with different CPU capabilities. The deliberately throttled Node 2 (0.5 CPU) did not degrade overall cluster performance under hardware-aware routing, because the middleware dynamically reduced its traffic share.

7 Technology Stack

Table 4: Technology stack used in the project.

Component	Technology
LLM Engine	llama-cpp-python (GGUF format)
Model	TinyLlama-1.1B-Chat-v1.0 (Q4_K_M quantized)
Web Framework	FastAPI + Uvicorn
HTTP Client	httpx (async), aiohttp (benchmarks)
System Monitoring	psutil
Container Runtime	Docker, Docker Compose
Orchestration	Kubernetes (Docker Desktop)
Visualization	matplotlib, seaborn, pandas, numpy
Scripting	Bash (deployment/cleanup automation)

8 Project Structure

Listing 3: Repository layout

```

Group22_GRS_Project/
|-- docker-compose.yaml      # Local orchestration
|-- requirements.txt          # Python dependencies
|-- group22_inference_service/
|   |-- Dockerfile           # Multi-stage build
|   |-- group22_app.py        # FastAPI inference node
|-- group22_load_balancer/
|   |-- Dockerfile
|   |-- group22_config.py     # Plug-and-play node list
|   |-- group22_middleware.py # Routing logic
|   |-- group22_telemetry.py  # HTTP telemetry poller
|-- group22_k8s/
|   |-- inference-cluster.yaml # 3 heterogeneous pods
|   |-- load-balancer.yaml     # LB deployment + service
|-- group22_benchmarks/
|   |-- group22_load_generator.py # Async load tool
|   |-- group22_analyze_results.py # KPI analysis
|   |-- group22_plot_results.py  # Visualization
|-- group22_scripts/
|   |-- group22_deploy_remote.sh # One-click deploy
|   |-- group22_cleanup_remote.sh # Full cleanup
|-- group22_results/          # Benchmark JSON outputs
|-- group22_plots/            # Generated PNG plots

```

9 How to Run

9.1 Prerequisites

- Docker & Docker Compose installed.
- Python 3.10+ with `pip install -r requirements.txt`.
- (For K8s) `kubectl` configured with a running cluster.
- TinyLlama GGUF model placed in `./models/` directory.

9.2 Local Deployment

```
# Start all containers
docker-compose up --build -d

# Run benchmark (normal load, hardware-aware)
python group22_benchmarks/group22_load_generator.py \
    --url http://localhost:8080 --strategy ha --load normal

# Generate plots
python group22_benchmarks/group22_plot_results.py
```

9.3 Remote Kubernetes Deployment

```
# One-click deployment to remote host
bash group22_scripts/group22_deploy_remote.sh

# Cleanup
bash group22_scripts/group22_cleanup_remote.sh
```

10 Conclusion

This project demonstrates that **hardware-aware load balancing** is not merely an optimization but a **necessity** for distributed LLM inference on heterogeneous CPU clusters. The key contributions are:

- A modular, plug-and-play architecture that separates routing policy from telemetry collection.
- A lightweight burden-score formula combining CPU utilization, memory pressure, and active-request count.
- Comprehensive benchmarking showing $5.5\times$ latency reduction and 100% vs. 30% success rates under stress.
- Automated deployment scripts for both local Docker Compose and remote Kubernetes environments.

The results validate that real-time resource monitoring, combined with congestion-aware routing, enables reliable inference even when individual nodes are resource-constrained. Future work could extend this approach to GPU-based clusters, implement predictive (rather than reactive) routing, and explore auto-scaling policies based on the telemetry data.

References

1. llama-cpp-python: <https://github.com/abetlen/llama-cpp-python>
2. TinyLlama: <https://huggingface.co/TheBloke/TinyLlama-1.1B-Chat-v1.0-GGUF>
3. FastAPI: <https://fastapi.tiangolo.com/>
4. Kubernetes Documentation: <https://kubernetes.io/docs/>
5. psutil: <https://psutil.readthedocs.io/>